

Nama : Deswita Khansa Rafifah

NIM : 254107020151

Kelas : TI-1G

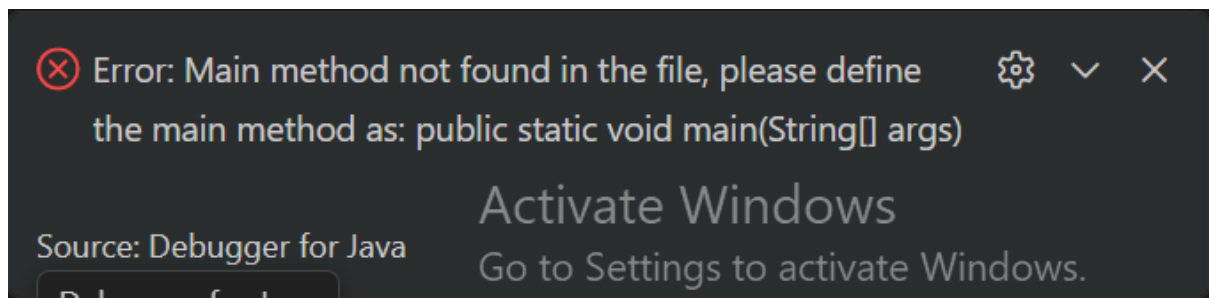
LAPORAN JOBSHEET 2

Percobaan 1: Deklarasi Class, Atribut dan Method

Kode program:

```
File Edit Selection View Go ...  
J Mahasiswa06.java X J MahasiswaMain06.java A  
Jobsheet2 > J Mahasiswa06.java > J Mahasiswa06 > tampilkanInformasi()  
3 public class Mahasiswa06 {  
5     String nama;  
6     String nim;  
7     String kelas;  
8     double ipk;  
9  
10 void tampilkanInformasi() {  
11     System.out.println("Nama: " + nama);  
12     System.out.println("NIM: " + nim);  
13     System.out.println("IPK: " + ipk);  
14     System.out.println("Kelas: " + kelas);  
15 }  
16  
17 void ubahKelas (String kelasBaru) {  
18     kelas = kelasBaru;  
19 }  
20  
21 void updateIpk (double ipkBaru) {  
22     ipk = ipkBaru;  
23 }  
24  
25 String nilaiKinerja() {  
26     if (ipk >= 3.5) {  
27         return "Kinerja sangat baik";  
28     } else if (ipk >= 3.0) {  
29         return "Kinerja Baik";  
30     } else if (ipk >= 2.0) {  
31         return "Kinerja Cukup";  
32     } else {  
33  
34     }  
35 }  
36 }  
37  
38 }  
39 }  
40 }  
41 }  
42 }  
43 }  
44 }  
45 }  
46 }  
47 }  
48 }  
49 }  
50 }  
51 }  
52 }  
53 }  
54 }  
55 }  
56 }  
57 }  
58 }  
59 }  
60 }  
61 }  
62 }  
63 }  
64 }  
65 }  
66 }  
67 }  
68 }  
69 }  
70 }  
71 }  
72 }  
73 }  
74 }  
75 }  
76 }  
77 }  
78 }  
79 }  
80 }  
81 }  
82 }  
83 }  
84 }  
85 }  
86 }  
87 }  
88 }  
89 }  
90 }  
91 }  
92 }  
93 }  
94 }  
95 }  
96 }  
97 }  
98 }  
99 }  
100 }  
101 }  
102 }  
103 }  
104 }  
105 }  
106 }  
107 }  
108 }  
109 }  
110 }  
111 }  
112 }  
113 }  
114 }  
115 }  
116 }  
117 }  
118 }  
119 }  
120 }  
121 }  
122 }  
123 }  
124 }  
125 }  
126 }  
127 }  
128 }  
129 }  
130 }  
131 }  
132 }  
133 }  
134 }  
135 }  
136 }  
137 }  
138 }  
139 }  
140 }  
141 }  
142 }  
143 }  
144 }  
145 }  
146 }  
147 }  
148 }  
149 }  
150 }  
151 }  
152 }  
153 }  
154 }  
155 }  
156 }  
157 }  
158 }  
159 }  
160 }  
161 }  
162 }  
163 }  
164 }  
165 }  
166 }  
167 }  
168 }  
169 }  
170 }  
171 }  
172 }  
173 }  
174 }  
175 }  
176 }  
177 }  
178 }  
179 }  
180 }  
181 }  
182 }  
183 }  
184 }  
185 }  
186 }  
187 }  
188 }  
189 }  
190 }  
191 }  
192 }  
193 }  
194 }  
195 }  
196 }  
197 }  
198 }  
199 }  
200 }  
201 }  
202 }  
203 }  
204 }  
205 }  
206 }  
207 }  
208 }  
209 }  
210 }  
211 }  
212 }  
213 }  
214 }  
215 }  
216 }  
217 }  
218 }  
219 }  
220 }  
221 }  
222 }  
223 }  
224 }  
225 }  
226 }  
227 }  
228 }  
229 }  
230 }  
231 }  
232 }  
233 }  
234 }  
235 }  
236 }  
237 }  
238 }  
239 }  
240 }  
241 }  
242 }  
243 }  
244 }  
245 }  
246 }  
247 }  
248 }  
249 }  
250 }  
251 }  
252 }  
253 }  
254 }  
255 }  
256 }  
257 }  
258 }  
259 }  
260 }  
261 }  
262 }  
263 }  
264 }  
265 }  
266 }  
267 }  
268 }  
269 }  
270 }  
271 }  
272 }  
273 }  
274 }  
275 }  
276 }  
277 }  
278 }  
279 }  
280 }  
281 }  
282 }  
283 }  
284 }  
285 }  
286 }  
287 }  
288 }  
289 }  
290 }  
291 }  
292 }  
293 }  
294 }  
295 }  
296 }  
297 }  
298 }  
299 }  
300 }  
301 }  
302 }  
303 }  
304 }  
305 }  
306 }  
307 }  
308 }  
309 }  
310 }  
311 }  
312 }  
313 }  
314 }  
315 }  
316 }  
317 }  
318 }  
319 }  
320 }  
321 }  
322 }  
323 }  
324 }  
325 }  
326 }  
327 }  
328 }  
329 }  
330 }  
331 }  
332 }  
333 }  
334 }  
335 }  
336 }  
337 }  
338 }  
339 }  
340 }  
341 }  
342 }  
343 }  
344 }  
345 }  
346 }  
347 }  
348 }  
349 }  
350 }  
351 }  
352 }  
353 }  
354 }  
355 }  
356 }  
357 }  
358 }  
359 }  
360 }  
361 }  
362 }  
363 }  
364 }  
365 }  
366 }  
367 }  
368 }  
369 }  
370 }  
371 }  
372 }  
373 }  
374 }  
375 }  
376 }  
377 }  
378 }  
379 }  
380 }  
381 }  
382 }  
383 }  
384 }  
385 }  
386 }  
387 }  
388 }  
389 }  
390 }  
391 }  
392 }  
393 }  
394 }  
395 }  
396 }  
397 }  
398 }  
399 }  
400 }  
401 }  
402 }  
403 }  
404 }  
405 }  
406 }  
407 }  
408 }  
409 }  
410 }  
411 }  
412 }  
413 }  
414 }  
415 }  
416 }  
417 }  
418 }  
419 }  
420 }  
421 }  
422 }  
423 }  
424 }  
425 }  
426 }  
427 }  
428 }  
429 }  
430 }  
431 }  
432 }  
433 }  
434 }  
435 }  
436 }  
437 }  
438 }  
439 }  
440 }  
441 }  
442 }  
443 }  
444 }  
445 }  
446 }  
447 }  
448 }  
449 }  
450 }  
451 }  
452 }  
453 }  
454 }  
455 }  
456 }  
457 }  
458 }  
459 }  
460 }  
461 }  
462 }  
463 }  
464 }  
465 }  
466 }  
467 }  
468 }  
469 }  
470 }  
471 }  
472 }  
473 }  
474 }  
475 }  
476 }  
477 }  
478 }  
479 }  
480 }  
481 }  
482 }  
483 }  
484 }  
485 }  
486 }  
487 }  
488 }  
489 }  
490 }  
491 }  
492 }  
493 }  
494 }  
495 }  
496 }  
497 }  
498 }  
499 }  
500 }  
501 }  
502 }  
503 }  
504 }  
505 }  
506 }  
507 }  
508 }  
509 }  
510 }  
511 }  
512 }  
513 }  
514 }  
515 }  
516 }  
517 }  
518 }  
519 }  
520 }  
521 }  
522 }  
523 }  
524 }  
525 }  
526 }  
527 }  
528 }  
529 }  
530 }  
531 }  
532 }  
533 }  
534 }  
535 }  
536 }  
537 }  
538 }  
539 }  
540 }  
541 }  
542 }  
543 }  
544 }  
545 }  
546 }  
547 }  
548 }  
549 }  
550 }  
551 }  
552 }  
553 }  
554 }  
555 }  
556 }  
557 }  
558 }  
559 }  
560 }  
561 }  
562 }  
563 }  
564 }  
565 }  
566 }  
567 }  
568 }  
569 }  
570 }  
571 }  
572 }  
573 }  
574 }  
575 }  
576 }  
577 }  
578 }  
579 }  
580 }  
581 }  
582 }  
583 }  
584 }  
585 }  
586 }  
587 }  
588 }  
589 }  
590 }  
591 }  
592 }  
593 }  
594 }  
595 }  
596 }  
597 }  
598 }  
599 }  
600 }  
601 }  
602 }  
603 }  
604 }  
605 }  
606 }  
607 }  
608 }  
609 }  
610 }  
611 }  
612 }  
613 }  
614 }  
615 }  
616 }  
617 }  
618 }  
619 }  
620 }  
621 }  
622 }  
623 }  
624 }  
625 }  
626 }  
627 }  
628 }  
629 }  
630 }  
631 }  
632 }  
633 }  
634 }  
635 }  
636 }  
637 }  
638 }  
639 }  
640 }  
641 }  
642 }  
643 }  
644 }  
645 }  
646 }  
647 }  
648 }  
649 }  
650 }  
651 }  
652 }  
653 }  
654 }  
655 }  
656 }  
657 }  
658 }  
659 }  
660 }  
661 }  
662 }  
663 }  
664 }  
665 }  
666 }  
667 }  
668 }  
669 }  
670 }  
671 }  
672 }  
673 }  
674 }  
675 }  
676 }  
677 }  
678 }  
679 }  
680 }  
681 }  
682 }  
683 }  
684 }  
685 }  
686 }  
687 }  
688 }  
689 }  
690 }  
691 }  
692 }  
693 }  
694 }  
695 }  
696 }  
697 }  
698 }  
699 }  
700 }  
701 }  
702 }  
703 }  
704 }  
705 }  
706 }  
707 }  
708 }  
709 }  
710 }  
711 }  
712 }  
713 }  
714 }  
715 }  
716 }  
717 }  
718 }  
719 }  
720 }  
721 }  
722 }  
723 }  
724 }  
725 }  
726 }  
727 }  
728 }  
729 }  
730 }  
731 }  
732 }  
733 }  
734 }  
735 }  
736 }  
737 }  
738 }  
739 }  
740 }  
741 }  
742 }  
743 }  
744 }  
745 }  
746 }  
747 }  
748 }  
749 }  
750 }  
751 }  
752 }  
753 }  
754 }  
755 }  
756 }  
757 }  
758 }  
759 }  
760 }  
761 }  
762 }  
763 }  
764 }  
765 }  
766 }  
767 }  
768 }  
769 }  
770 }  
771 }  
772 }  
773 }  
774 }  
775 }  
776 }  
777 }  
778 }  
779 }  
780 }  
781 }  
782 }  
783 }  
784 }  
785 }  
786 }  
787 }  
788 }  
789 }  
790 }  
791 }  
792 }  
793 }  
794 }  
795 }  
796 }  
797 }  
798 }  
799 }  
800 }  
801 }  
802 }  
803 }  
804 }  
805 }  
806 }  
807 }  
808 }  
809 }  
810 }  
811 }  
812 }  
813 }  
814 }  
815 }  
816 }  
817 }  
818 }  
819 }  
820 }  
821 }  
822 }  
823 }  
824 }  
825 }  
826 }  
827 }  
828 }  
829 }  
830 }  
831 }  
832 }  
833 }  
834 }  
835 }  
836 }  
837 }  
838 }  
839 }  
840 }  
841 }  
842 }  
843 }  
844 }  
845 }  
846 }  
847 }  
848 }  
849 }  
850 }  
851 }  
852 }  
853 }  
854 }  
855 }  
856 }  
857 }  
858 }  
859 }  
860 }  
861 }  
862 }  
863 }  
864 }  
865 }  
866 }  
867 }  
868 }  
869 }  
870 }  
871 }  
872 }  
873 }  
874 }  
875 }  
876 }  
877 }  
878 }  
879 }  
880 }  
881 }  
882 }  
883 }  
884 }  
885 }  
886 }  
887 }  
888 }  
889 }  
890 }  
891 }  
892 }  
893 }  
894 }  
895 }  
896 }  
897 }  
898 }  
899 }  
900 }  
901 }  
902 }  
903 }  
904 }  
905 }  
906 }  
907 }  
908 }  
909 }  
910 }  
911 }  
912 }  
913 }  
914 }  
915 }  
916 }  
917 }  
918 }  
919 }  
920 }  
921 }  
922 }  
923 }  
924 }  
925 }  
926 }  
927 }  
928 }  
929 }  
930 }  
931 }  
932 }  
933 }  
934 }  
935 }  
936 }  
937 }  
938 }  
939 }  
940 }  
941 }  
942 }  
943 }  
944 }  
945 }  
946 }  
947 }  
948 }  
949 }  
950 }  
951 }  
952 }  
953 }  
954 }  
955 }  
956 }  
957 }  
958 }  
959 }  
960 }  
961 }  
962 }  
963 }  
964 }  
965 }  
966 }  
967 }  
968 }  
969 }  
970 }  
971 }  
972 }  
973 }  
974 }  
975 }  
976 }  
977 }  
978 }  
979 }  
980 }  
981 }  
982 }  
983 }  
984 }  
985 }  
986 }  
987 }  
988 }  
989 }  
990 }  
991 }  
992 }  
993 }  
994 }  
995 }  
996 }  
997 }  
998 }  
999 }  
1000 }
```

Hasil Running:



Pertanyaan

1. Sebutkan dua karakteristik class atau object!

Jawab:

Dua karakteristik class atau object adalah:

1. Mempunyai sesuatu (atribut) yaitu data atau properti yang dimiliki oleh object. Contoh pada class Mahasiswa: nim, nama, kelas, ipk.
2. Melakukan sesuatu (method) yaitu fungsi atau tingkah laku yang dapat dilakukan oleh object. Contoh pada class Mahasiswa: tampilkanInformasi(), updateIpk().

2. Perhatikan class Mahasiswa pada Praktikum 1 tersebut, ada berapa atribut yang dimiliki oleh class Mahasiswa? Sebutkan apa saja atributnya!

Jawab:

Class Mahasiswa memiliki 4 atribut, yaitu:

- nim (String)
- nama (String)
- kelas (String)
- ipk (double)

3. Ada berapa method yang dimiliki oleh class tersebut? Sebutkan apa saja methodnya!

Jawab:

Class Mahasiswa memiliki 4 method, yaitu:

1. tampilkanInformasi()
2. ubahKelas(String: kelasBaru)
3. updateIpk(double: ipkBaru)
4. nilaiKinerja()

4. Perhatikan method updateIpk() yang terdapat di dalam class Mahasiswa. Modifikasi isi method tersebut sehingga IPK yang dimasukkan valid yaitu terlebih dahulu dilakukan pengecekan apakah IPK yang dimasukkan di dalam rentang 0.0 sampai dengan 4.0 ($0.0 \leq \text{IPK} \leq 4.0$). Jika IPK tidak pada rentang tersebut maka dikeluarkan pesan: "IPK tidak valid. Harus antara 0.0 dan 4.0".

Jawab:

```
void updateIpk (double ipkBaru) {  
    if (ipkBaru >= 0.0 && ipkBaru <= 4.0) {  
        ipk = ipkBaru;  
    } else {  
        System.out.println(x: "IPK tidak valid. Harus antara 0.0 dan 4.0");  
    }  
}
```

5. Jelaskan bagaimana cara kerja method nilaiKinerja() dalam mengevaluasi kinerja mahasiswa, kriteria apa saja yang digunakan untuk menentukan nilai kinerja tersebut, dan apa yang dikembalikan (di-return-kan) oleh method nilaiKinerja() tersebut?

Jawab:

Method nilaiKinerja(double ipk) bekerja dengan struktur percabangan if-else bertingkat:

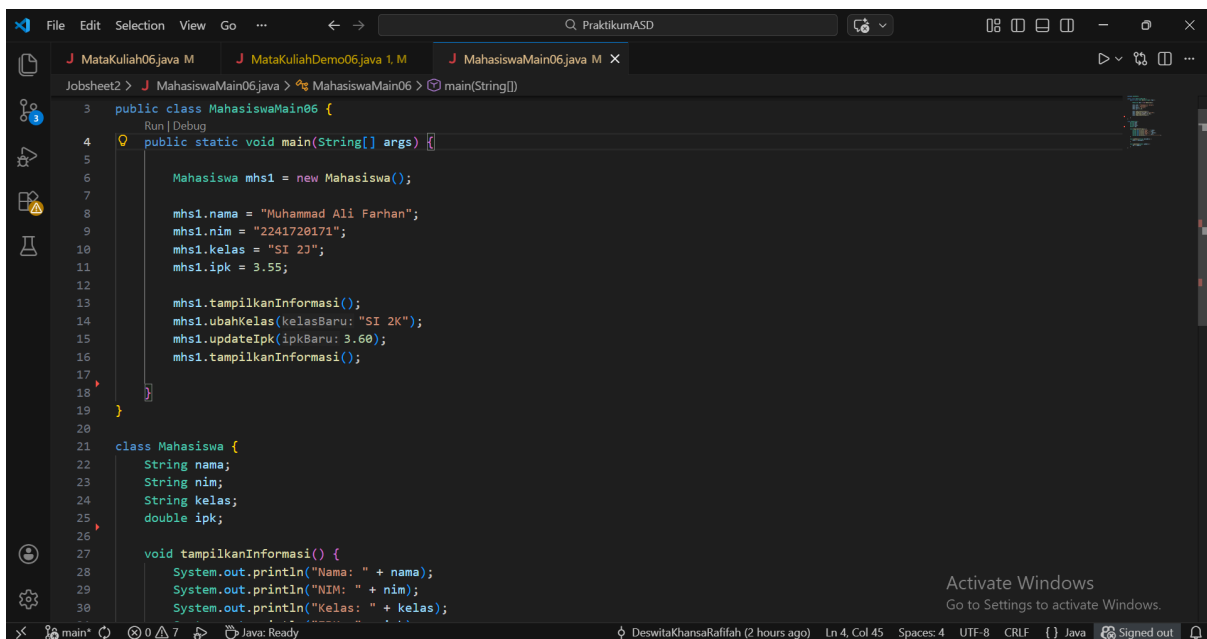
- Parameter: menerima nilai ipk bertipe double sebagai input.
- Kriteria evaluasi dan nilai yang dikembalikan:
 - $\text{ipk} \geq 3.5 \rightarrow \text{return "Kinerja sangat baik"}$
 - $\text{ipk} \geq 3.0 \rightarrow \text{return "Kinerja baik"}$
 - $\text{ipk} \geq 2.0 \rightarrow \text{return "Kinerja cukup"}$

- $ipk < 2.0 \rightarrow \text{return "Kinerja kurang"}$

Return value: Method ini mengembalikan nilai bertipe String berisi kategori kinerja. Percabangan dievaluasi dari atas ke bawah — kondisi pertama yang terpenuhi langsung dikembalikan dan kondisi berikutnya tidak diperiksa lagi. Kondisi $ipk < 2.0$ ditangani oleh blok else terakhir tanpa perlu menulis kondisi eksplisit.

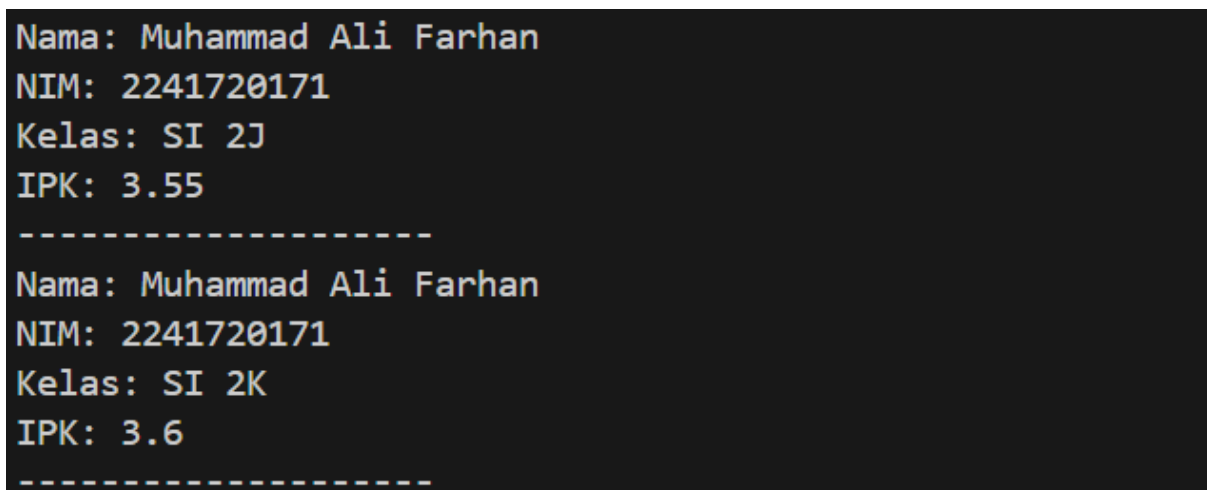
Percobaan 2: Instansiasi Object, serta Mengakses Atribut dan Method

Kode Program:



```
File Edit Selection View Go ...
MataKuliah06.java M MataKuliahDemo06.java 1, M MahasiswaMain06.java M x
Jobsheet2 > J MahasiswaMain06.java > % MahasiswaMain06 > main(String[])
3 public class MahasiswaMain06 {
4     public static void main(String[] args) {
5
6         Mahasiswa mhs1 = new Mahasiswa();
7
8         mhs1.nama = "Muhammad Ali Farhan";
9         mhs1.nim = "2241720171";
10        mhs1.kelas = "SI 2J";
11        mhs1.ipk = 3.55;
12
13        mhs1.tampilkanInformasi();
14        mhs1.ubahKelas(kelasBaru: "SI 2K");
15        mhs1.updateIpk(ipkBaru: 3.60);
16        mhs1.tampilkanInformasi();
17    }
18 }
19
20
21 class Mahasiswa {
22     String nama;
23     String nim;
24     String kelas;
25     double ipk;
26
27     void tampilkanInformasi() {
28         System.out.println("Nama: " + nama);
29         System.out.println("NIM: " + nim);
30         System.out.println("Kelas: " + kelas);
31     }
32 }
```

Hasil Running:



```
Nama: Muhammad Ali Farhan
NIM: 2241720171
Kelas: SI 2J
IPK: 3.55
-----
Nama: Muhammad Ali Farhan
NIM: 2241720171
Kelas: SI 2K
IPK: 3.6
-----
```

Pertanyaan

1. Pada class MahasiswaMain, tunjukkan baris kode program yang digunakan untuk proses instansiasi! Apa nama object yang dihasilkan?

Jawab:

- Baris kode instansiasi → Mahasiswa mhs1 = new Mahasiswa();
- Penjelasan:
 - Mahasiswa → nama class
 - mhs1 → nama object
 - new → keyword untuk membuat object
 - Mahasiswa() → konstruktor

Proses ini disebut **instansiasi**, yaitu proses pembuatan object dari class. Object yang dihasilkan bernama **mhs1**.

2. Bagaimana cara mengakses atribut dan method dari suatu objek?

Jawab: Atribut dan method diakses menggunakan operator titik (.) setelah nama object. Contoh: mhs1.nama untuk atribut atau mhs1.tampilkanInformasi() untuk method.

3. Mengapa hasil output pemanggilan method tampilkanInformasi() pertama dan kedua berbeda?

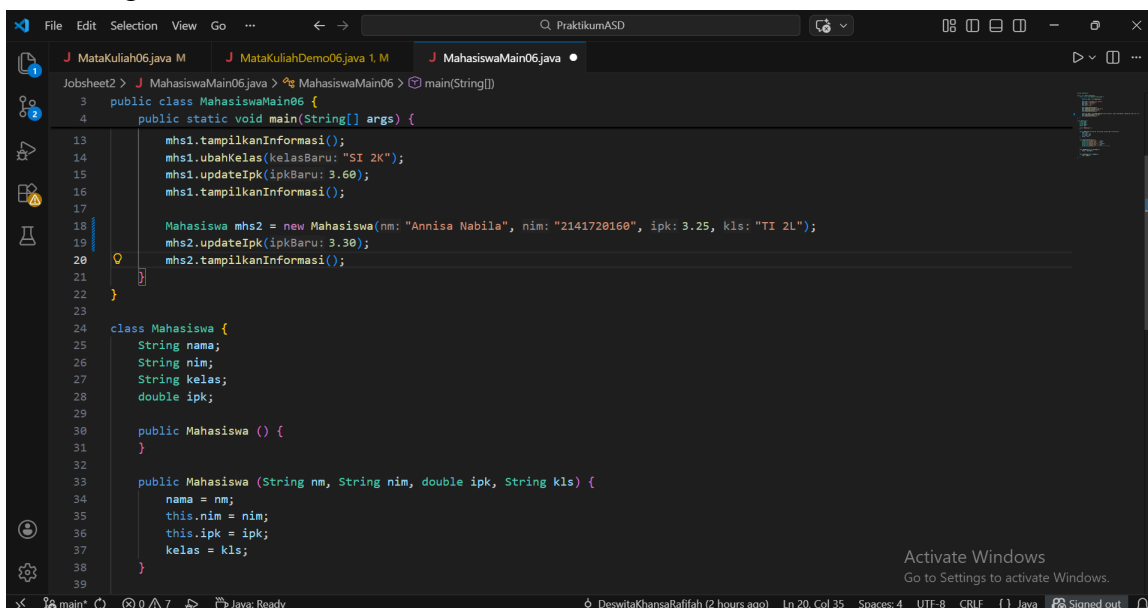
Jawab: Karena di antara pemanggilan tampilkanInformasi() pertama dan kedua, terdapat dua pemanggilan method yang mengubah nilai atribut (state) dari object mhs1:

1. mhs1.ubahKelas("SI 2K") → mengubah atribut kelas dari "SI 2J" menjadi "SI 2K"
2. mhs1.updateIpk(3.60) → mengubah atribut ipk dari 3.55 menjadi 3.6

Sehingga saat tampilkanInformasi() dipanggil kedua kalinya, nilai yang ditampilkan adalah nilai atribut yang sudah diperbarui, bukan nilai awal.

Percobaan 3: Membuat Konstruktor

Kode Program:



```
File Edit Selection View Go ... Q PraktikumASD
J MataKuliah06.java M J MataKuliahDemo06.java 1.M J MahasiswaMain06.java
Jobsheet2 > J MahasiswaMain06.java > % MahasiswaMain06 > main(String[])
3 public class MahasiswaMain06 {
4     public static void main(String[] args) {
13         mhs1.tampilkanInformasi();
14         mhs1.ubahKelas(kelasBaru: "SI 2K");
15         mhs1.updateIpk(ipkBaru: 3.60);
16         mhs1.tampilkanInformasi();
17
18         Mahasiswa mhs2 = new Mahasiswa(nm: "Annisa Nabila", nim: "2141720160", ipk: 3.25, kls: "TI 2L");
19         mhs2.updateIpk(ipkBaru: 3.30);
20         mhs2.tampilkanInformasi();
21
22     }
23
24     class Mahasiswa {
25         String nama;
26         String nim;
27         String kelas;
28         double ipk;
29
30         public Mahasiswa () {
31
32
33         public Mahasiswa (String nm, String nim, double ipk, String kls) {
34             nama = nm;
35             this.nim = nim;
36             this.ipk = ipk;
37             kelas = kls;
38
39     }
40 }
```

Hasil Running:

```
Nama: Muhammad Ali Farhan
NIM: 2241720171
Kelas: SI 2J
IPK: 3.55
-----
Nama: Muhammad Ali Farhan
NIM: 2241720171
Kelas: SI 2K
IPK: 3.6
-----
Nama: Annisa Nabila
NIM: 2141720160
Kelas: TI 2L
IPK: 3.3
-----
```

Pertanyaan

1. Pada class Mahasiswa di Percobaan 3, tunjukkan baris kode program yang digunakan untuk mendeklarasikan konstruktor berparameter!

Jawab:

```
public Mahasiswa (String nm, String nim, double ipk, String kls) {
    nama = nm;
    this.nim = nim;
    this.ipk = ipk;
    kelas = kls;
}
```

Konstruktor ini memiliki 4 parameter: nm (nama), nim, ipk, dan kls (kelas). Kata kunci this digunakan pada this.nim dan this.ipk karena nama parameter sama persis dengan nama atribut class, sehingga this diperlukan untuk merujuk ke atribut milik class, bukan parameter lokal.

2. Perhatikan class MahasiswaMain. Apa sebenarnya yang dilakukan pada baris program berikut?

```
Mahasiswa mhs2 = new Mahasiswa("Annisa Nabila", "2141720160", 3.25, "TI 2L");
```

Jawab: Baris tersebut melakukan instansiasi sekaligus inisialisasi object baru bernama mhs2 dari class Mahasiswa menggunakan konstruktor berparameter. Proses yang terjadi:

- Kata kunci new mengalokasikan memori untuk object Mahasiswa baru.
- Konstruktor berparameter dipanggil dengan 4 argumen yang langsung diassign ke atribut:
 - "Annisa Nabila" → atribut nama
 - "2141720160" → atribut nim
 - 3.25 → atribut ipk
 - "TI 2L" → atribut kelas

Hasilnya object mhs2 langsung memiliki nilai pada semua atributnya saat instansiasi, tanpa perlu pengisian satu per satu seperti pada konstruktor default.

3. Hapus konstruktor default pada class Mahasiswa, kemudian compile dan run program. Bagaimana hasilnya? Jelaskan mengapa hasilnya demikian!

Jawab:

- Hasil: Error pada saat kompilasi di baris Mahasiswa mhs1 = new Mahasiswa();.
- Alasan: Jika kita mendefinisikan konstruktor berparameter secara manual, Java tidak akan lagi menyediakan konstruktor default secara otomatis. Oleh karena itu, pemanggilan new Mahasiswa() tanpa parameter akan dianggap ilegal.

4. Setelah melakukan instansiasi object, apakah method di dalam class Mahasiswa harus diakses secara berurutan? Jelaskan alasannya!

Jawab: Tidak. Method dapat dipanggil dalam urutan apapun tergantung kebutuhan logika program, selama objectnya sudah diinstansiasi terlebih dahulu. Namun ada hal yang perlu diperhatikan:

- Jika menggunakan konstruktor default, atribut perlu diisi sebelum method yang menggunakan atribut tersebut dipanggil.
- Jika satu method bergantung pada hasil method lain (misalnya nilaiKinerja() dipanggil setelah updateIpk() agar hasilnya sesuai nilai terbaru), maka urutan menjadi penting secara logika.
- Jika tidak ada ketergantungan antar method, urutan pemanggilan bebas dan tidak akan mempengaruhi hasil.

5. Buat object baru dengan nama mhs<NamaMahasiswa> menggunakan konstruktor berparameter dari class Mahasiswa!

Jawab:

```
Mahasiswa mhs2 = new Mahasiswa(nm: "Deswita Khansa Rafifah", nim: "254107020151", ipk: 3.70, kls: "TI 16");  
mhs2.tampilkanInformasi();
```